# 50. Reinforcement Learning

*"Learn through trial, error, and reward. Until the solution is no longer guessed, but understood."*

**Previous Section:**

49. Multi-Layer Perceptrons

**Contents:**



You have come this far, which means you are standing near the end of this journey. And yes, this topic is dense, but it is also one of the most fascinating ideas in all of Machine Learning.

Reinforcement Learning can be seen as a branch of Machine Learning, but it also feels like its own world. Unlike everything you have learned so far, this is not about predicting labels, fitting lines, or grouping data. It introduces something new: An **agent, a**n **environment,** and a **goal driven by reward.**

The agent does not learn from labeled data. It does not learn from clean examples. Instead, it learns by interacting with the world. It takes an action, observes what happens, and receives feedback in the form of reward or

penalty. That is, **a good decision leads to a reward**, and a bad **decision leads to a penalty.** And through this continuous cycle, the agent slowly improves.

At the beginning, it knows nothing. Its actions are uncertain, sometimes even completely wrong. But it keeps trying. It explores. It fails. It adjusts. And over time, it begins to understand which actions lead to better outcomes.

This is what makes Reinforcement Learning different. It is not learning from answers. It is learning from consequences. And the ultimate objective is simple, yet powerful:

> ***Find a strategy that maximizes the total reward over time.***

Not perfect immediately. Not correct at every step. But progressively better, until the behavior itself becomes intelligent.

## 1. What is Reinforcement Learning?

Reinforcement Learning is where learning stops being passive, and becomes an experience.

Instead of sitting in front of a dataset and trying to predict answers, we introduce something that can act. An agent. And this agent is placed into a world, an environment, where nothing is given for free. No labels. No correct answers. Only consequences.

The agent moves, makes decisions, and observes what happens next: If the outcome is good, it receives a reward. If the outcome is bad, it receives a penalty. And that is all it has to learn from.

At the beginning, it does not understand anything. Every action is just a guess. Sometimes it works. Most of the time, it does not. But it keeps going. Step by step, action by action, it begins to connect decisions with outcomes. Not instantly, but gradually. Quietly building a strategy.

This is the key difference.

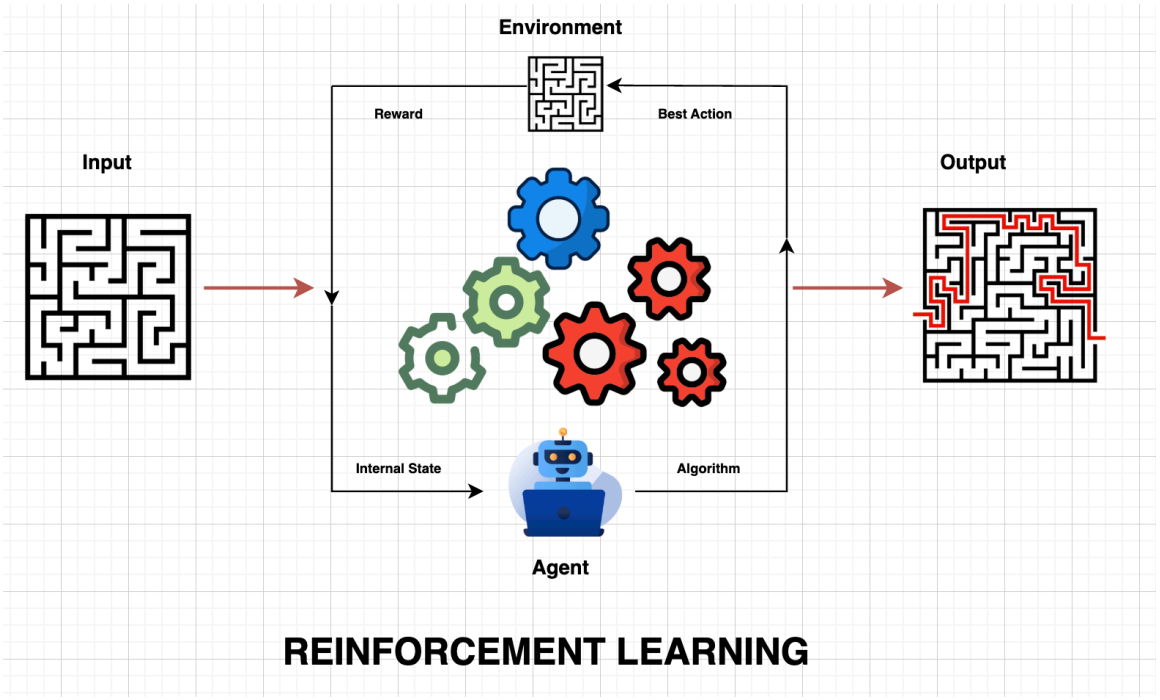
- Supervised Learning teaches through answers
- Unsupervised Learning searches for patterns
- Reinforcement Learning learns through consequences

And because of that, the structure of the data changes completely. Instead of independent rows in a table, everything becomes a sequence:

- A **state** — where the agent is
- An **environment** - the world or system the agent interacts with
- An **action** — what it decides to do
- A **reward** — what it receives afterward

Each step depends on the previous one. Every decision shapes the next situation. Learning is no longer isolated. It becomes a continuous flow.

*(You can reference the image below for the core idea of Reinforcement Learning)*




You can think of it like a student, but not the kind that studies from books.

- In supervised learning, the student practices with solutions already provided
- In unsupervised learning, the student tries to organize problems without guidance
- In semi-supervised learning, the student learns from a few examples and generalizes
- In reinforcement learning, the student plays the game. No answer sheet. No instructions. Just moves, outcomes, and experience.

- At first, the student loses. A lot.
- But over time, something changes. They begin to recognize which decisions lead to better positions. Which strategies fail. Which paths are worth repeating.
- And eventually, without ever being told the “correct answer,” they learn how to act.

That is Reinforcement Learning. Not learning what is right, but discovering it.

## 2. Key Concepts in Reinforcement Learning

Reinforcement Learning is not just about an agent acting randomly and hoping for the best. There is a structure behind every decision it makes. A quiet system that guides how it observes, reacts, and improves over time. Let’s break that structure down.

### Core Components of Reinforcement Learning

Every Reinforcement Learning system is built on a few fundamental pieces. You can think of them as the agent’s way of understanding the world.

- **Policy:** This is the agent’s behavior. A rule, or sometimes a very complex function, that answers one question: *“What should I do right now?”.* Given a situation, the policy decides the action. A self-driving car sees a pedestrian → it stops. That decision comes from its policy.
- **Reward Signal:** This is the only feedback the agent truly understands. It tells the agent whether an action was good or bad. Positive rewards push behavior forward. Penalties push it away. The agent is not told *why* something is correct. It only feels the outcome.
- **Value Function:** Not all rewards are immediate. Some decisions look good now but fail later. Others seem slow but lead to better outcomes. The value function helps the agent think ahead. It estimates how good a situation is in the long run, not just in the next step. It is the difference between acting fast, and acting wisely.
- **Model:** Some agents learn purely from experience. Others try to imagine the future.
  A model allows the agent to predict what might happen next. If I do this,

what will the environment do? It turns learning into planning. Not just reacting, but anticipating.

### How Reinforcement Learning Works

At its core, Reinforcement Learning is a loop. A continuous interaction between the agent and the environment.

- The agent observes its current situation (state)
- It chooses an action based on its policy
- The environment responds with a new state and a reward
- The agent updates what it knows
- And then, it does it again

This cycle repeats thousands, sometimes millions of times. But there is something subtle happening inside this loop. The agent is always balancing two behaviors:

- **Exploration** — trying new actions to discover better possibilities
- **Exploitation** — using what it already knows works well

Too much exploration, and it never settles. Too much exploitation, and it might miss something better. Somewhere in between, is learning.

This entire process is often described using something called a **Markov Decision Process (MDP)**. Which simply means: the next step depends only on where you are now and what you choose to do. Not on the entire past. Just the present moment.

### Types of Reinforcement

Not all feedback works the same way. The way rewards are given shapes how the agent behaves.

- **Positive Reinforcement:** A reward is given after a good action. The agent learns: *“Do this again.”*
    - Strength: Encourages growth and consistent improvement
    - Weakness: Too many rewards can lose their meaning over time

- **Negative Reinforcement:** An unpleasant condition is removed when the agent behaves correctly. The agent learns: *"Avoid this outcome."*
    - Strength: Encourages meeting basic expectations
    - Weakness: The agent may only do the minimum required

The difference is subtle, but important. One pulls behavior forward. The other pushes it away from failure.

### Types of Reinforcement Learning Approaches

Finally, not all agents learn in the same way. It depends on how they experience the world.

- **Online Reinforcement Learning:** The agent learns by interacting directly with the environment. Every action generates new experience. Learning happens in real time. Mistakes are part of the process.
- **Offline Reinforcement Learning (Batch RL):** The agent does not explore the environment itself. Instead, it learns from past experiences collected by others. Logs, demonstrations, historical data. It studies before it acts.

### Model-Based or Model-Free

**Both online and offline reinforcement learning can be either model-based or model-free**. The distinction between online and offline refers to *how the data is collected*, whereas the distinction between model-based and model-free refers to *how the agent uses that data to make decisions*

- **Model-Based:** The agent builds an internal representation (a "model") of the environment's dynamics—predicting how the world works, including what the next state and reward will be given any action.
- **Model-Free:** The agent does not try to model or understand the environment's mechanics. Instead, it learns directly by trial and error, mapping states directly to actions or estimating the expected rewards of taking specific actions.
- **Online Reinforcement Learning:** The agent interacts directly with the environment, generating experiences in real time. This can be **model-free** (e.g., using algorithms like Q-learning or SARSA to learn optimal

actions) or **model-based** (e.g., using the Dynamic architecture, where the agent learns a model of the environment and uses it to plan future moves).

- **Offline Reinforcement Learning (Batch RL):** The agent learns purely from a static, pre-collected dataset of past experiences without further exploration. This can also be **model-free** (e.g., Batch Constrained Q-learning) or **model-based** (e.g., learning a transition model directly from the logged dataset and planning within that estimated environment).

Reinforcement Learning, in the end, is not just about maximizing reward. It is about building behavior. An agent that starts with no understanding, learns to act, learns to anticipate. And eventually, learns to choose not just what works. But what works best over time.

## 3. Reinforcement Learning - The Maze Game

Let me show you how all of this comes together. Not as theory, but as something you can actually see. Imagine a maze. A simple grid. Like a matrix.

- The top-left corner is **(0, 0)**
- The goal is at **(9, 9)**
- Some cells are open paths
- Some cells are walls, places you cannot step into

And inside this maze, there is an agent. A mouse. At the beginning, the mouse is placed somewhere in the maze. It does not know where the goal is. It does not know which paths are blocked. To it, everything is just, unknown.

**The Setup**

- **Agent**: The mouse
- **Environment**: The maze
- **State**: The mouse's current position (for example, (2, 3))
- **Actions**: Move up, down, left, or right
- **Reward**:
  - +10 for reaching the goal

    - 1 for each step (to encourage efficiency)
    - 5 if it hits a wall

**What Happens at the Beginning**

At first, the mouse moves randomly. It tries to go right → hits a wall. It goes up → moves forward. It goes left → back to where it started.

There is no intelligence yet. Just trial and error. But every move gives feedback.

- Hitting a wall feels bad → penalty
- Moving freely feels neutral
- Reaching the goal feels very good → reward

And slowly, patterns begin to form.

**The Internal State (What the Mouse Learns)**

Here is where things become interesting. The mouse is not just moving. It is **building an internal understanding of the maze**. Not visually. Not like we do. But numerically. Inside, it starts forming something like a **value map**:

- *"If I am at (1,1)... this position is not very useful"*
- *"If I am at (8,8)... I am close to something good"*
- *"If I go right from here... that usually leads to a wall"*

It begins to assign **values to states** and **preferences to actions**.

This is its internal state. Not memory in the human sense. But learned experience. You can think of it like this:

- Each position in the maze gets a score
- Each action gets a likelihood of being chosen
- These numbers keep updating after every step

At first, the values are random. Meaningless. But over time:

- Paths that lead closer to the goal gain higher value
- Dead ends become less attractive
- Walls become clearly avoidable

**From Random Moves to Strategy**

At some point, something changes. The mouse is no longer wandering. It avoids walls without needing to try them again. It chooses paths that seem promising. It reaches the goal faster, and more consistently. It has not memorized the maze perfectly. It has learned **how to navigate it**. That is the difference.

Reinforcement Learning is not about giving the mouse a map. It is about letting the mouse **discover the map by living inside it**.

Every mistake becomes information. Every reward becomes direction. Every step updates its internal understanding. And eventually, without ever being told the correct path, the mouse finds it. Not once. But reliably. That is Reinforcement Learning.

## 4. Reinforcement Learning with Python

Now, let's turn everything into something real. Not just concepts. Not just imagination. We are going to build the maze. We are going to place the mouse inside it. And we are going to let it learn.

**Step 1: The Maze (The Environment)**

We represent the maze as a matrix.

- **0** → free path (the agent can walk)
- **1** → wall (blocked)

```
import numpy as np

maze = np.array([[0, 1, 1, 1, 1, 1, 1, 1, 1, 1],
                 [0, 0, 0, 0, 1, 0, 0, 0, 0, 1],
                 [1, 1, 1, 0, 1, 0, 1, 1, 0, 1],
                 [1, 0, 0, 0, 0, 0, 1, 0, 0, 1],
                 [1, 0, 1, 1, 1, 1, 1, 0, 1, 1],
                 [1, 0, 1, 0, 0, 0, 0, 0, 1, 1],
                 [1, 0, 1, 0, 1, 1, 1, 0, 1, 1],
                 [1, 0, 1, 0, 1, 0, 0, 0, 1, 1],
```

```
                  [1, 0, 1, 0, 1, 0, 1, 0, 0, 1],
                  [1, 1, 1, 0, 1, 1, 1, 1, 0, 0]])

start = (0, 0)
goal = (9, 9)
```

The mouse starts at **(0, 0)**. The goal is **(9, 9)**. Everything else, it must discover.

### Step 2: The Agent's Memory (Q-Table)

The agent stores what it learns in a **Q-table**.

- Each **state** = a position in the maze
- Each **action** = up, down, left, right
- Each value = "how good is this action here?"

```
actions = [(0, -1), (0, 1), (-1, 0), (1, 0)]  # left, righ
t, up, down
Q = np.zeros(maze.shape + (len(actions),))
```

At the beginning, all values are zero. Meaning the agent knows nothing.

### Step 3: Learning Parameters

These control *how* the agent learns.

```
num_episodes = 5000
alpha = 0.1      # learning rate
gamma = 0.9      # future importance
epsilon = 0.5    # exploration

reward_fire = -10
reward_goal = 50
reward_step = -1
```

- **alpha** → how fast it updates
- **gamma** → how much it cares about the future

- **epsilon** → how often it explores randomly

Rewards shape behavior:

- Hit wall → bad
- Step → small penalty
- Reach goal → big reward

### Step 4: Helper Functions

Check if a move is valid, and choose actions.

```
def is_valid(pos):
    r, c = pos
    if r < 0 or r >= maze.shape[0]:
        return False
    if c < 0 or c >= maze.shape[1]:
        return False
    if maze[r, c] == 1:
        return False
    return True

def choose_action(state):
    if np.random.random() < epsilon:
        return np.random.randint(len(actions))  # explore
    else:
        return np.argmax(Q[state])  # exploit
```

This is the balance:

- Sometimes it **explores**
- Sometimes it **trusts what it learned**

### Step 5: Learning Loop (Q-Learning)

This is where everything happens.

```
rewards_all_episodes = []
```

```
for episode in range(num_episodes):
    state = start
    total_rewards = 0
    done = False

    while not done:
        action_index = choose_action(state)
        action = actions[action_index]

        next_state = (state[0] + action[0], state[1] + acti
on[1])

        if not is_valid(next_state):
            reward = reward_fire
            done = True
        elif next_state == goal:
            reward = reward_goal
            done = True
        else:
            reward = reward_step

        old_value = Q[state][action_index]
        next_max = np.max(Q[next_state]) if is_valid(next_s
tate) else 0

        Q[state][action_index] = (old_value + alpha *
                                  (reward + gamma * next_ma
x - old_value))

        state = next_state
        total_rewards += reward

    epsilon = max(0.01, epsilon * 0.995)
    rewards_all_episodes.append(total_rewards)
```

What is happening here? The agent moves → Receives reward → Updates its belief (Q-table) → And repeats, thousands of times.

This is the equation behind it:

$$Q(s,a) = Q(s,a) + \alpha \cdot (reward + \gamma \cdot \max Q(next) - Q(s,a))$$

Do not overcomplicate it. It simply means: "*Adjust what I thought... based on what just happened.*"

#### Step 6: Extract the Learned Path

Once learning is done, we follow the best actions.

```
def get_optimal_path(Q, start, goal, actions, maze, max_ste
ps=200):
    path = [start]
    state = start
    visited = set()

    for _ in range(max_steps):
        if state == goal:
            break

        visited.add(state)
        best_action = None
        best_value = -float('inf')

        for idx, move in enumerate(actions):
            next_state = (state[0] + move[0], state[1] + mo
ve[1])

            if (0 <= next_state[0] < maze.shape[0] and
                0 <= next_state[1] < maze.shape[1] and
                maze[next_state] == 0 and
                next_state not in visited):

                if Q[state][idx] > best_value:
                    best_value = Q[state][idx]
                    best_action = idx

        if best_action is None:
```

```
            break

        move = actions[best_action]
        state = (state[0] + move[0], state[1] + move[1])
        path.append(state)

    return path

optimal_path = get_optimal_path(Q, start, goal, actions, ma
ze)
```

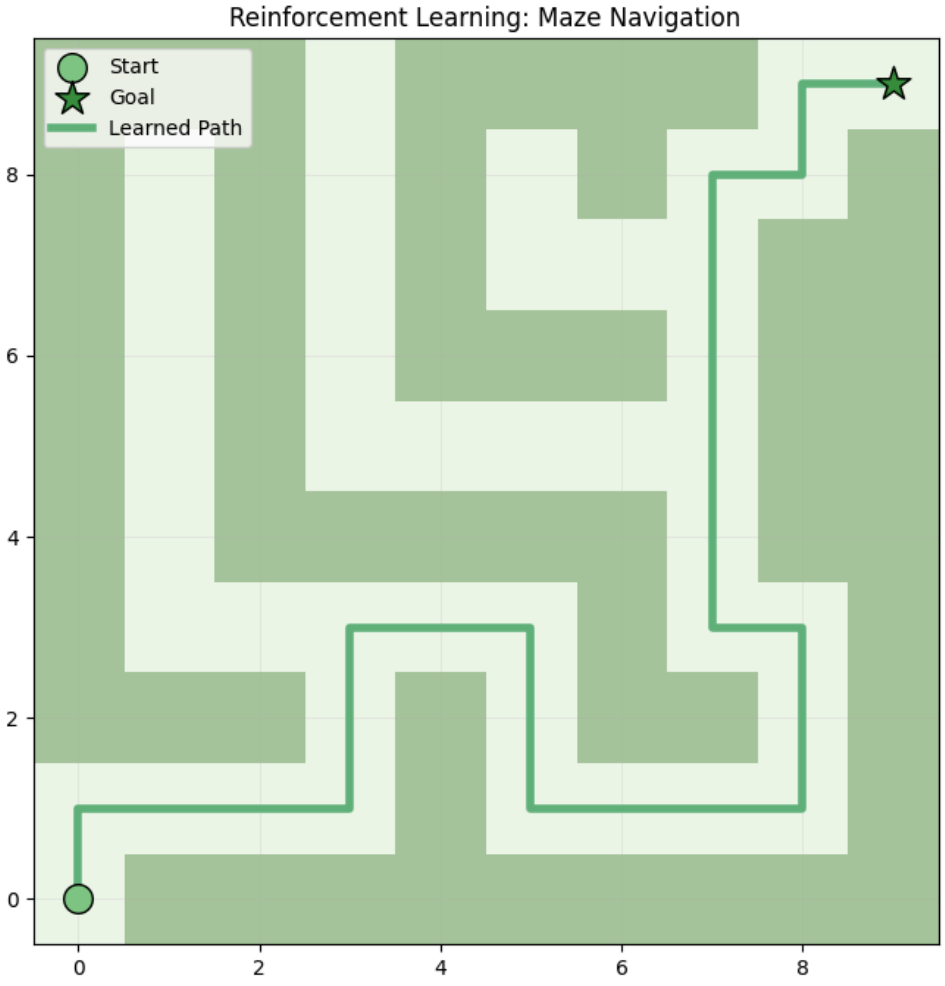
This is no longer learning. This is **execution**. Following the best decisions the agent discovered.

### Step 7: Visualize the Result

Now, we see what the agent has learned.

```
def plot_maze_with_path(path):
    cmap = ListedColormap(['#eef8ea', '#a8c79c'])
    plt.figure(figsize=(8, 8))
    plt.imshow(maze, cmap=cmap)

    plt.scatter(start[1], start[0], marker='o', color='#81c
784',
                edgecolors='black', s=200, label='Start', z
order=5)

    plt.scatter(goal[1], goal[0], marker='*', color='#388e3
c',
                edgecolors='black', s=300, label='Goal', zo
rder=5)

    rows, cols = zip(*path)
    plt.plot(cols, rows, color='#60b37a', linewidth=4,
             label='Learned Path', zorder=4)

    plt.title('Reinforcement Learning: Maze Navigation')
```

```
    plt.gca().invert_yaxis()
    plt.grid(True, alpha=0.2)
    plt.legend()
    plt.show()

plot_maze_with_path(optimal_path)
```




**What Just Happened?** At the beginning: The agent was lost. It hit walls. It wandered randomly. At the end: It found a path. It avoided mistakes. It moved with purpose

And the most important part: **We never told it the solution.** It learned it. Step by step. Mistake by mistake. Reward by reward. That is Reinforcement Learning at its finest.

## 5. State–Action–Reward–State–Action (SARSA)

Reinforcement Learning is not just about finding the best action. It is about **learning while acting**. And SARSA is one of the clearest examples of that idea.

**What makes SARSA different?** Most algorithms try to learn the *best possible action* at the next step. SARSA does something more honest. It learns from **what the agent actually does next**. Not the optimal action. Not the perfect choice. But the *real* one. That is why we call it **on-policy learning**. It follows its own behavior, and improves it at the same time.

**The Flow of SARSA**

The name itself tells the story:

- **State (S)** → where the agent is
- **Action (A)** → what it decides to do
- **Reward (R)** → what it receives
- **Next State (S′)** → where it ends up
- **Next Action (A′)** → what it chooses next

And then, it learns from that entire sequence. Not just a single step. But the transition between decisions.

**The Update Rule -** At the core, SARSA updates its understanding using:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[ r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right]$$

You can read it simply like this: ***"Update what I thought... based on what I actually did next."***

- $\alpha$ **(alpha)** → how fast we learn
- $\gamma$ **(gamma)** → how much we care about the future

The key difference:

- Q-Learning looks at the *best possible next action*
- SARSA looks at the *actual next action taken*

So SARSA learns more cautiously. More realistically.

**SARSA in the Maze Game (Python) -** Now, let's return to the maze. Same environment. Same agent. But this time, it learns using SARSA.

- **Epsilon-Greedy Policy**

```
import numpy as np
import random

def epsilon_greedy_policy(Q, state, epsilon):
    if random.uniform(0, 1) < epsilon:
        return random.randint(0, 3)  # explore
    else:
        return np.argmax(Q[state[0], state[1]])  # exploit
```

Simple idea: Sometimes guess. Sometimes trust experience

- **SARSA Algorithm**

```
def sarsa(env, episodes, alpha, gamma, epsilon):
    Q = np.zeros((env.height, env.width, 4))  # 4 actions

    for episode in range(episodes):
        state = env.reset()
        action = epsilon_greedy_policy(Q, state, epsilon)

        done = False

        while not done:
            next_state, reward, done = env.step(action)

            next_action = epsilon_greedy_policy(Q, next_sta
te, epsilon)

            Q[state[0], state[1], action] += alpha * (rewar
d + gamma * Q[next_state[0], next_state[1], next_action]
                                                    - Q[s
tate[0], state[1], action])
```

```
            state = next_state
            action = next_action

    return Q
```

Look carefully: *The agent chooses an action → Moves to the next state → Chooses the next action → Then updates using* ***that exact action.***

It is learning from its own behavior. Not an ideal version of itself. But the *real one*.

In the maze:

- Q-Learning says: *"What is the best possible move next?"*
- SARSA says: *"What did I actually do next?"*

That small difference changes everything. SARSA becomes more **conservative**. More **realistic**. Sometimes slower, but safer. It does not chase perfection. It learns from experience. Step by step. Action by action. Just like the mouse, still inside the maze.

## 6. Q-Learning

There is a moment in Reinforcement Learning where the agent stops asking: "What did I just do?". And starts asking: "What *should* I have done?". That moment is Q-Learning.

**What makes Q-Learning different?**

Q-Learning does not follow the agent's current behavior. It looks beyond it. It learns from the **best possible action in the future**, even if the agent did not take it.

That is why we call it **off-policy**. SARSA learns from *experience, while* Q-Learning learns from *potential.* It is more ambitious. Sometimes more aggressive. But often faster at finding the optimal path.

**The Core Idea -** The agent builds a **Q-table:**

- Each **state** → a position in the maze
- Each **action** → a possible move

- Each value → "How good is this action here in the long run?"

At the beginning, these values mean nothing. But through interaction, they evolve.

### How Q-Learning Updates Knowledge

At every step, the agent adjusts its understanding using:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[ r + \gamma \max_{a} Q(s', a) - Q(s,a) \right]$$

Read it simply: *"Update what I thought... using the best possible future outcome."*

- $\alpha$ **(alpha)** → how fast it learns
- $\gamma$ **(gamma)** → how much it values the future
- $\max Q(s', a)$ → the best action it *could* take next

This is the key difference from SARSA. Q-Learning does not ask: *"What did I do next?".* It asks: *"What is the best thing I could do next?"*

### How the Learning Process Works

The loop is simple, but powerful:

- Observe current state → Choose action (ε-greedy) → Execute action → Receive reward and next state → Update Q-value → Repeat

And across many episodes, something happens. The Q-table begins to reflect reality. Not perfectly at first. But increasingly accurate.

### Exploration vs Exploitation

Just like before, balance is everything.

- **Exploration (**$\varepsilon$**)** → try random moves
- **Exploitation (**$1 - \varepsilon$**)** → use learned knowledge

Without exploration → stuck in local paths; Without exploitation → never improves. Q-Learning walks between the two.

### Q-Learning Function

Now, let's bring it back to the maze. Same environment. Same goal. But this time, the agent learns using Q-Learning.

```
def q_learning(env, episodes, alpha, gamma, epsilon):
    Q = np.zeros((env.height, env.width, 4))  # 4 actions

    for episode in range(episodes):
        state = env.reset()
        done = False

        while not done:
            action = epsilon_greedy_policy(Q, state, epsilo
n)
            next_state, reward, done = env.step(action)
            best_next = np.max(Q[next_state[0], next_state
[1]])

            Q[state[0], state[1], action] += alpha * (rewar
d + gamma * best_next
                                                      - Q[s
tate[0], state[1], action])

            state = next_state

    return Q
```

Look closely at the update step: The agent takes an action → It observes what happens → But instead of following its next action. It asks: "What is the best possible move from here?". And uses that to update its knowledge.

Back in the maze: SARSA learns cautiously, following its own path. Q-Learning learns optimistically, chasing the best path So SARSA is safer, more realistic; but Q-Learning is faster, more ideal. And in many cases, Q-Learning reaches the goal sooner. Not because it experienced the perfect path. But because it

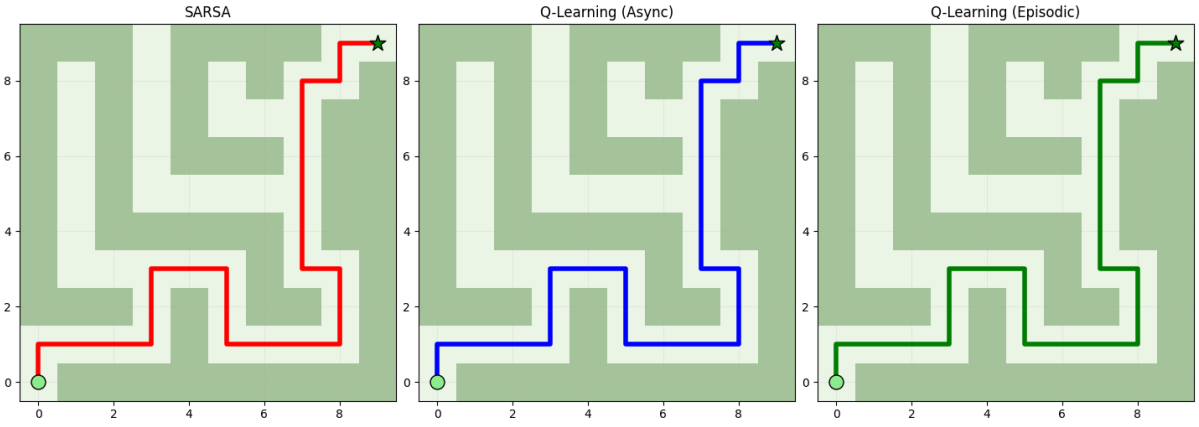
**imagined it, and learned toward it**. That is the essence of Q-Learning. Not just learning from what is. But learning from what *could be*.

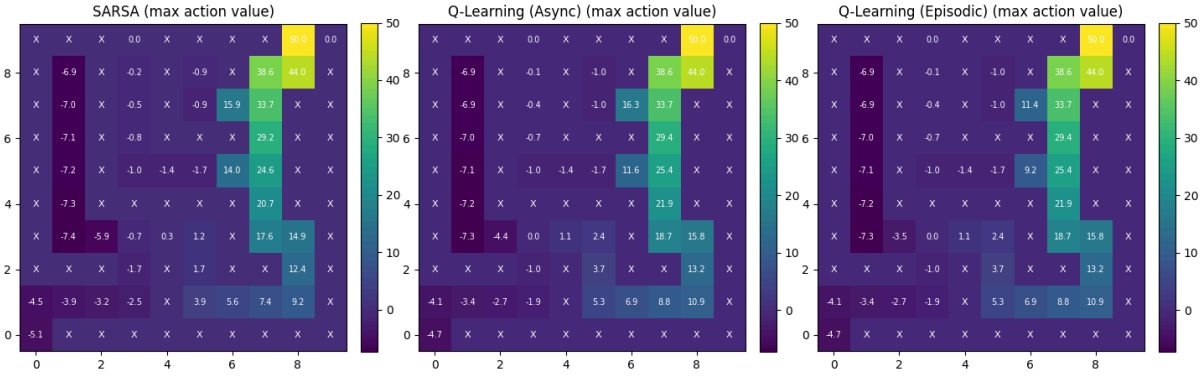
*Below is the full code that:*

- *Defines three learning methods: async Q-Learning (q_learning), episodic Q-Learning (episodic_q_learning), and SARSA (sarsa).*
- *Trains all three on the same maze.*
- *Draws three maze subplots (one per algorithm) showing learned greedy paths.*
- *Draws three Q-table heatmaps (max action-value per state) with numeric values; positive cells indicate states that lead (in expectation) toward the goal, large negative values correspond to fire/penalty or poor trajectories.*

reinforcement_learning.py

Expected outputs:






## Summary

Reinforcement Learning is perhaps the closest step we have toward decision-making intelligence inspired by game theory.  It is not about predicting answers. It is not about fitting models to data. It is about **learning how to act**.

An agent is placed into an environment with a goal, but no instructions. Every move it makes leads to a consequence. Good decisions are rewarded. Mistakes are penalized. And through this process, the agent slowly improves.

You can think of it like playing a game. Like *Super Mario*. At the beginning, every jump is uncertain. The character falls, hits obstacles, loses progress. But each failure carries information. Over time, the player begins to anticipate, react, and choose better actions.

Reinforcement Learning works the same way. It does not start with knowledge. It builds it through experience. And the objective is simple: Find a strategy that maximizes reward over time. Not by being told the solution. But by discovering it, one step at a time.

## References

https://www.geeksforgeeks.org/machine-learning/what-is-reinforcement-learning/

https://www.geeksforgeeks.org/machine-learning/sarsa-reinforcement-learning/

https://www.geeksforgeeks.org/machine-learning/q-learning-in-python/

https://www.datacamp.com/tutorial/reinforcement-learning-python-introduction

https://www.ibm.com/think/topics/reinforcement-learning

**Next Section:**

- 51. Machine Learning Workflow